

Midterm Report – Library Management System

INFO-C451 – Logan Young

Implementation in SDLC

System implementation is typically the fourth of five phases in the systems development life cycle. First you must make a plan to see what problems need to be solved. Then, you have to make an analysis of the requirements of the new system and put together a logical model – just an outline of what the system will do. Then comes the design portion of the SDLC; a physical, working model of the system is created, and specification and documentation are delivered with it. Finally, there is implementation (with systems support after it). The system is fully constructed and implemented alongside the rest of the company's systems.

Implementation is what is being done here. The planning has been done, the analysis was completed, and the physical model of the Library Management System (LMS) has been constructed. Now, the system must be implemented into the work environment.

Chosen Development Methodology

The software development method chosen for this project is a general spiral methodology—iteration and user feedback are important. The LMS is meant to automate a lot of manual labor, but it's also important that users of the system can interact with it easily and efficiently. The graphical user interface, for instance, is a major part of the system that would need to be revised over time after surveying different demographics of users.

Planning Objectives and Strategies

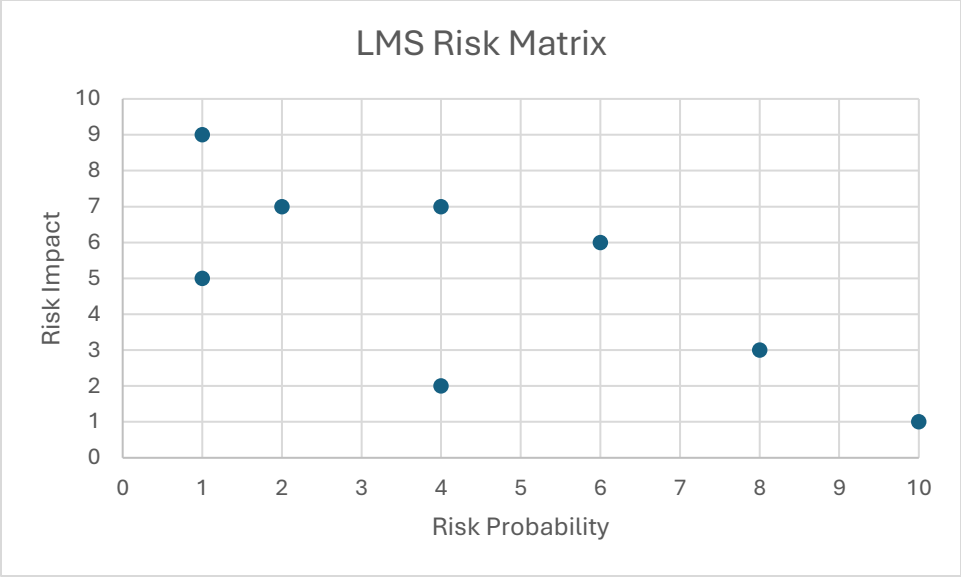
There are several key goals I'd like to accomplish during system implementation planning. First, I'd like to create a tight, detailed schedule, but one that also has room for unplanned delays. A schedule that does not count on any step being delayed is a schedule that will not survive development and implementation. Second, I'd like to create a risk identification, assessment, and management plan to help better inform the schedule. Next, of course, is dealing with the various intricacies of system deployment: data migration, testing, debugging, iteration, etc. Finally, I'd like to make a plan for system support after implementation is complete.

Risk Assessment

Risk is part of any development process, so it's important to account for anything that could come up, no matter how improbable. Thankfully, the Library Management System is far from complex and is not intrinsically urgent, so the probability and impact of certain risks are reduced compared to management systems in the healthcare or banking domains. Below are some examples of possible risks and a matrix for simple analysis. Risks are ranked on a scale from 1-10 with 1 being lowest in scale and

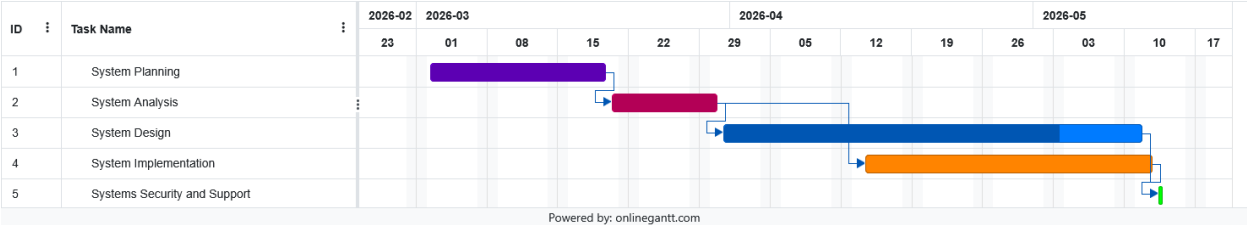
10 being highest in scale: a risk with an impact of 8 and a probability of 1 has a high impact, but a very low probability of occurring.

1. Development stalls due to extended worker issues. Since the team for creating the LMS is small, any worker shortage could cause issues. An artist quitting, or a programmer missing long periods of work could lead to delays. **Probability: 4, Impact: 7.**
2. In contrast to the previous issue, a single employee takes an extended break. They will undoubtedly return, but due to the small team, work might be slightly delayed. **Probability: 8, Impact: 3.**
3. Budget runs over due to lack of money. Libraries are often underfunded, so the costs of resources can become daunting. Running low on or out of money can be likely, unless extensive cost-cutting is implemented. **Probability: 6, Impact: 6.**
4. The physical server room is robbed or destroyed. The server room would be the only physical location (aside from the library itself) to worry about since development for the LMS would be done almost entirely remotely. Losing the servers itself would be a terrible loss of resources, but the room itself could be replaced. **Probability: 1, Impact: 5.**
5. Development tools become permanently unavailable (whether due to being taken down by the owners or licenses expiring). If open-source software is not used to create the LMS, this could prove to be disastrous during the design phase. Having to migrate tools could cause long delays. **Probability: 2, Impact: 7.**
6. Project files are temporarily lost, but backups remain in an offsite location. With good data storage practices, any single data loss will only be a short, temporary setback. **Probability: 4, Impact: 2.**
7. Project files and all backups are permanently lost. In this very unlikely scenario, design will have to begin again from scratch (with gained knowledge and experience). **Probability: 1, Impact: 9.**
8. Work on one aspect of the system stalls due to temporary roadblock. If a programmer finds a bug they cannot fix, or a hardware installer discovers there is no readily available power in the server room, work could very temporarily slow down. Any manner of obstruction could happen. **Probability: 10, Impact: 1.**



Project Timeline

The first four phases of the SDLC will take place over the course of two and a half working months. Implementation specifically will take approximately three weeks to complete. The Gantt chart below depicts the estimated progress of the project as the Systems Implementation phase is entered. The last phase—Systems Security and Support—has a duration of one day but will be continued indefinitely.



Installation Approach

For the system installation approach, I absolutely believe that a parallel operation changeover would be the best way to make the transition. As the library previously had no automated systems for management, both the library members and the library staff might have a difficult time transitioning from manual to automated. Directly cutting over would be too sudden a shift, and a pilot operation would not sufficiently stress test the new system (since we are using a spiral development approach). That leaves the phased and parallel operations as the two remaining installation choices. I believe parallel would be the superior choice because it would allow all users time to get used to the new system, and it would have a sufficient number of users for testing and feedback. Additionally, there aren't enough moving parts in the LMS to necessitate a phased operation changeover.

Installation Schedule

As mentioned before, installation of the LMS will take about three weeks. Below is an estimated schedule of activities that will have to be completed before installation is finished.

Days **1-14** – System is fully constructed based off of logical and physical models in design specifications.

Days **15-18** – Internal bug testing and QA are conducted, last minute changes are made.

Days **18-19** – Select server room in library and prepare room for hardware installation.

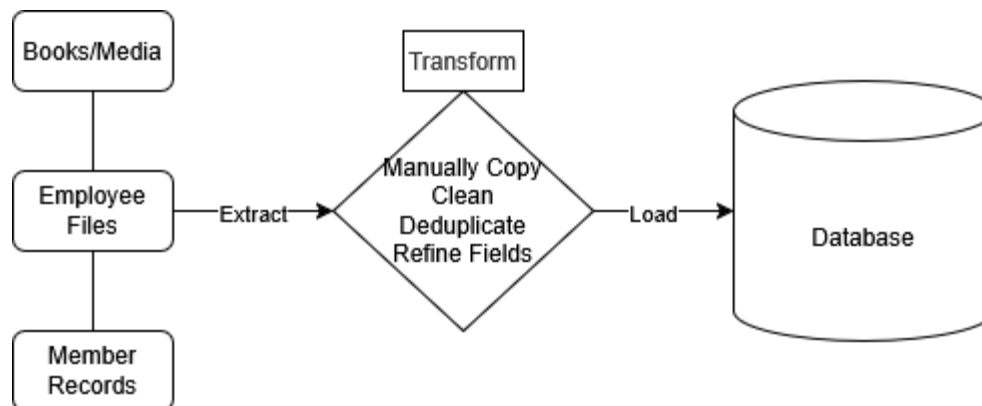
Day **20** – Install server architecture in server room.

Day **21** – Start server and begin access to the LMS.

Since a parallel operation approach is what's being used in this case, pen and paper records will continue to be used for a short while after installation so that the development team has time to collect feedback from users and modify the system to conform.

Data Migration Plan [ETL Diagram]

Once again, the fact that the previous system was pen, paper, and file proves to be a detriment for the new system implementation process. Rather than an easier transfer from digital to digital, data has to be manually copied from physical to digital. (Another possible solution would be to make photocopies/scans of all possible files and then use a machine learning algorithm to copy text and transfer to the database.) At least one person would have to copy all files into the new database, which could take quite a while depending on the size of the library. Data will have to be cleaned (due to human error), deduplicated (if there are duplicate records), and finally fields will have to be refined since physical databases can have some differences compared to digital ones.



Test Plan

Design a Test Plan outlining the scope, objectives, tools, and responsibilities.

Testing for the LMS will cover the usual four areas: unit, integration, system, and acceptance. Unit testing will be comprised of testing each individual function for the LMS to ensure all run without error given some test data (e.g., a login script, an "add book" function). Then, all connected functions and scripts will need to be tested together, likely by inserting test data into one function and then handing the results of that function to the next (like checking out a book for the logged in user and then changing that book to unavailable in the DB). System testing will occur when the previous two tests have finished: testers will run through a complete version of the site to ensure that everything works as

specified. Finally, acceptance testing will occur using the userbase as testers: library staff and members. If feedback is given on changes, these will be analyzed and possibly incorporated into the next iteration of the LMS. Most tools used will be IDEs or virtual environments; if needed, automatic tools like postman will be used to check data flows and ensure server connections are stable and working.

Test Cases

Create two or more detailed test cases (e.g., login validation, transaction processing).

Test Case ID	Description	Precondition	Input	Expected Output
TC-F-011	Member attempts to check out a book that is not available.	User account is member level or higher and chosen book is not available.	- Book: Animal Farm [Checked Out]	Book checkout fails, "This book is [Checked Out], please submit a hold request or choose another book!"
TC-V-004	Email does not comply with regex email check.	User has entered an incorrect email address into the email field.	- Email: lmsuser1.com	Account creation disabled, email field warning, "Email is invalid! Please check to make sure email is formatted correctly."

Debugging and Error Handling Strategies

Debugging must be done before the new system is deployed, but there is also room for user testing afterward. In other words, the system must be free of large, functional bugs before deployment. Different types of errors will need different strategies: server connection errors will require a review of the server architecture and functions that connect the client to the server to the database, functional errors will need a review of the application scripts, syntax errors typically point to themselves, etc. Using an IDE rather than a word processor makes error discovery and fixing much simpler.

Automated Testing Tools

As mentioned in the Test Plan, automatic tools will see limited usage, if any. The LMS is a conceptually and mechanically simple client-server application. Most testing and debugging can be done manually, saving on cost. Filling the database must also be handled manually, so it makes sense to clean and test the data manually too. Barring any unexpected sudden design changes, Postman would likely be the only automated tool to be used, for server connection testing purposes.

Data Quality Issues and Conversion Method

The main issue to grapple with here is the fact that data has to be transcribed from a physical format to a digital one. Whoever's filling in the database has to sit down and do the tedious job of reading from file row by row and then typing that into the DB, while also fixing capitalization, spelling errors, and duplication as they go (caused by human error when writing by file). Copying is also dependent on the handwriting of the files. What's more, filling the database by hand is also subject to

human error, so there should be at least two people to check each other's work. Aside from the aforementioned tasks, general cleaning will work by filling in empty fields or removing records for which there isn't enough information to fill in.

Testing and Validation Steps

Validation scripts can be written to test the newly filled database for its accuracy and integrity. For example: testing emails against a regex to ensure that they're structurally sound addresses. A program could also be written to extract all records with empty fields (and/or fields with nonstandard data types) for manual overview. The tester can then fill these fields using an outside data source if necessary (e.g., a member address without a zip code, a book title without an author).

Summary of System Implementation Plan

In summary, system implementation must go through six main steps:

1. System software construction from logical and physical models.
2. Hardware construction.
3. Software and hardware connection.
4. Data cleaning and copying.
5. System and data testing and validation.
6. System deployment and user testing.

All of these steps serve some important purpose such that system installation cannot proceed without all of the mentioned steps being completed.

Key Success Factors and Post-Implementation Review Criteria

System installation success will be measured against the specified needs outlined in the system plan. If all of the needs have been met (e.g., a system where a user can check out books, a system where an administrator can accept a staff employment application), then the system will have succeeded. Of course, plans and needs are ever-changing; the library staff could decide that they want a new feature halfway through development, and whether to create that feature now or after deployment must be decided on. The system should also be free of all functional and validation-related bugs; minor bugs like display issues on less commonly used web-connected devices can be handled during user testing (especially since the parallel operation system installation approach is being utilized).

Lessons Learned

The main lesson learned is that system implementation is much more difficult than it seems, perhaps being the most difficult and complex phase in the Software Development Life Cycle. Like most things though, software development is a continuous process, and experience (even failure) is a useful guide through this process. The development team may learn from a failure in hardware installation to allot more time to that step in the future, as well as to vet their hardware engineers better. They may learn from a sudden change in the client's requirements to make sure that their needs are more

explicitly understood. There's much to learn from on any project and adding lessons learned to the team's abilities helps future projects run smoother.